

Web Ontology Language

Elham

Introduction

- Web Ontology Language (OWL) is a Semantic Web language designed to represent rich and complex knowledge about things, groups of things, and relations between things.
- It is a knowledge representation language, designed to formulate, exchange, and reason with knowledge about a domain of interest (often called vocabulary).

How knowledge is represented in OWL?

The basic notions are:

- **Axioms:** the basic statements that an OWL ontology expresses
- **Entities:** elements used to refer to real-world objects
- **Expressions:** combinations of entities to form complex descriptions from basic ones

Having a closer look at statements in OWL

Axioms

- To formulate knowledge explicitly, it is useful to assume that it consists of elementary pieces that are often referred to as *statements* or *propositions*.
- Axioms are assumed to be always true.
- The way ontological axioms interact can be very subtle and difficult for people to understand.
- Example:
 - Mary is female
 - John and Mary are married

Entities

- All atomic constituents of statements are called *entities*.
 - objects (John, Mary)
 - categories (female)
 - relations (married)
- we denote objects as *individuals*, categories as *classes* and relations as *properties*.

Expressions

- Names of entities can be combined into *expressions* using so-called *constructors*.

Classes and Instances

- This statement talks about an individual named Mary and states that this individual is a person. Mary “is a member of” the *class* of all persons.

```
ClassAssertion( :Person :Mary )
```

- we can use the same type of statement to indicate that Mary is a woman by expressing that she is an instance of the class of women:

```
ClassAssertion( :Woman :Mary )
```

Object Properties

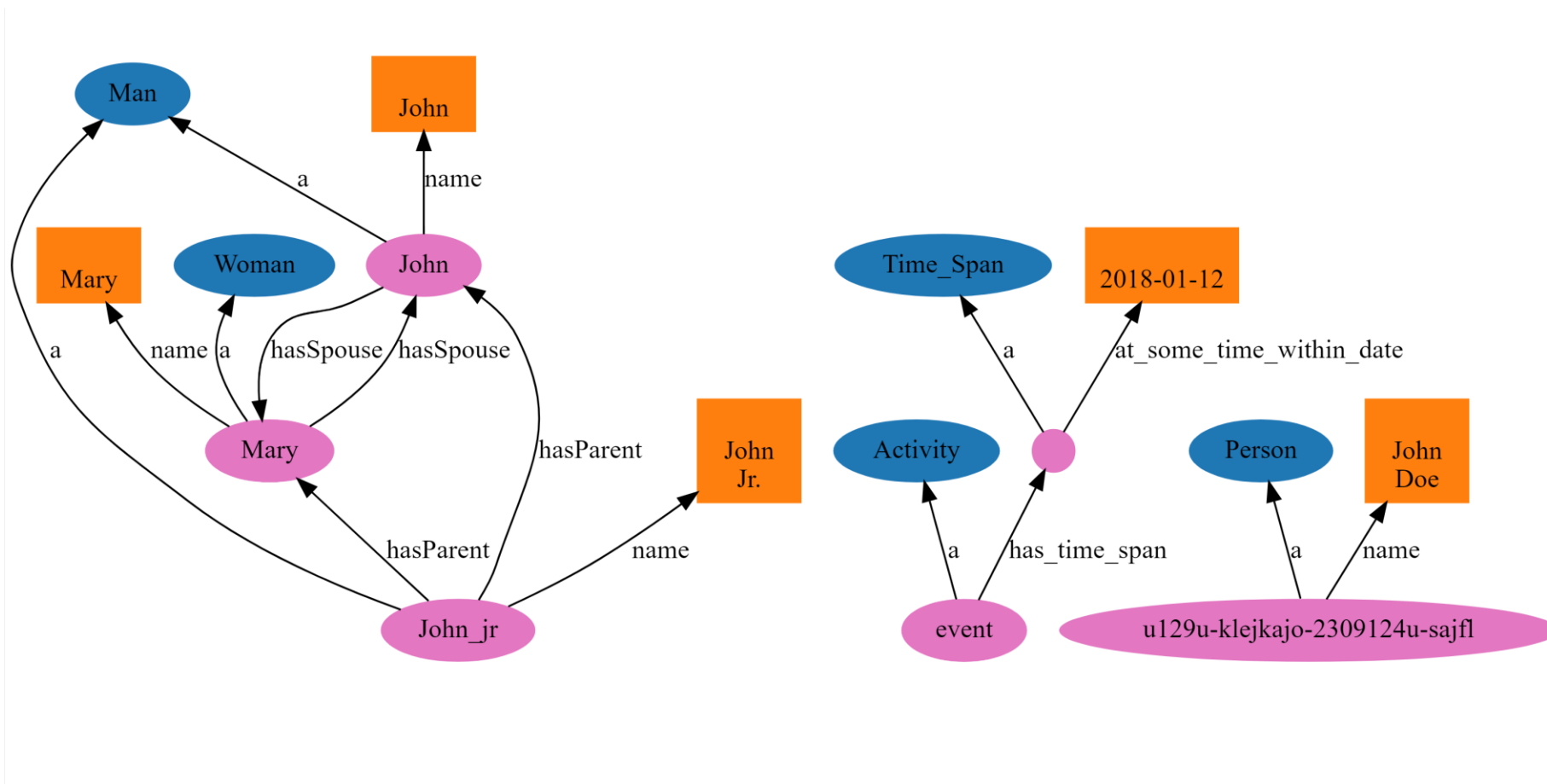
- An ontology is also meant to specify how the individuals relate to other individuals. For example: “Mary is John's wife”

```
ObjectPropertyAssertion( :hasWife :John :Mary )
```

- The entities describing in which way the individuals are related – like hasWife in our case, are called *properties*.


```
@prefix : <http://dig.isi.edu/> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

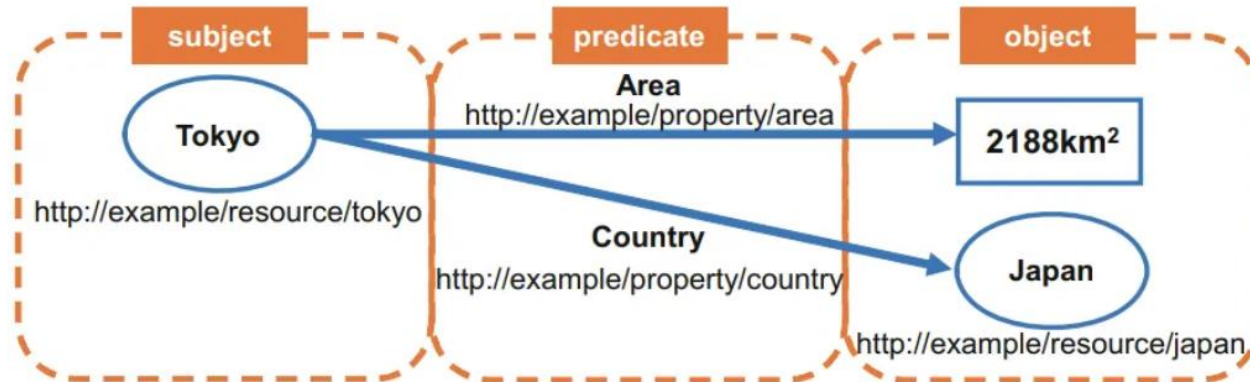
:John a :Man ;
    :name "John" ;
    :hasSpouse :Mary .
:Mary a :Woman ;
    :name "Mary" ;
    :hasSpouse :John .
:John_jr a :Man ;
    :name "John Jr." ;
    :hasParent :John, :Mary .
:Time_Span a owl:Class .
:event a :Activity ;
    :has_time_span [
        a :Time_Span ;
        :at_some_time_within_date "2018-01-12"^^xsd:date
    ] .
:u129u-klejkajo-2309124u-sajfl a :Person ;
    :name "John Doe" .
```



Understanding Linked Data Formats

Introduction

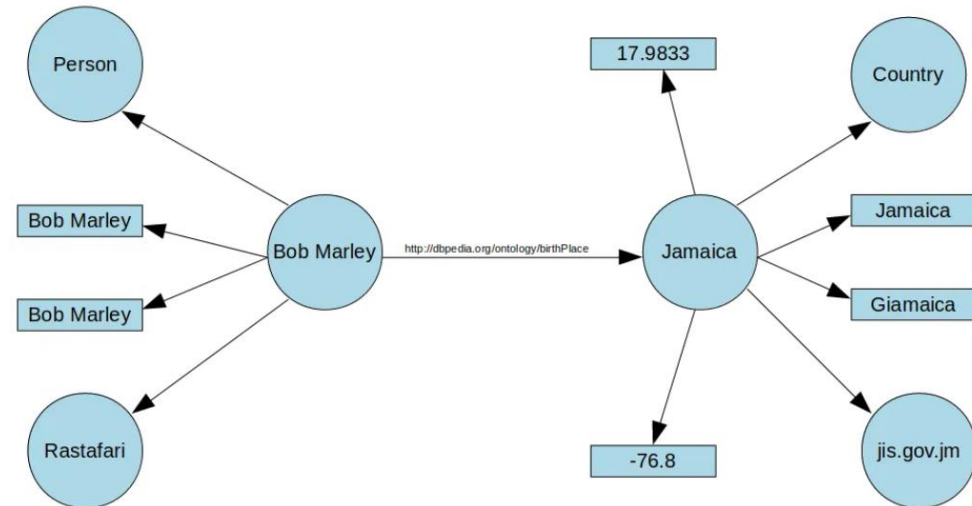
- Very simply, linked data is stored as triples that consist of a subject, predicate, and object.
- These triples connect entities to other entities to create a directed knowledge graph:



- RDF is commonly stored in one of four formats: N-Triples (.nt), Turtle (.ttl), JSON-LD (.json) or RDF/XML (.rdf).

N-Triples

- Storing and reading RDF as N-Triples is simple as every line of a .nt file is a single triple (<subject> <predicate> <object>) that together form a directed knowledge graph:
- circles represent entities,
- rectangles represent literals
- arrows illustrate the predicates



N-Triples format

Here we have 11 triples containing information about Bob Marley and Jamaica extracted from DBpedia

is of type Persois we use the foaf (friend of a friend) ontology as the URI for this type

```
(1) <http://dbpedia.org/resource/Bob_Marley> <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> <http://xmlns.com/foaf/0.1/Person> .
(2) <http://dbpedia.org/resource/Bob_Marley> <http://www.w3.org/2000/01/rdf-schema#label> "Bob Marley"@en .
(3) <http://dbpedia.org/resource/Bob_Marley> <http://www.w3.org/2000/01/rdf-schema#label> "Bob Marley"@fr .
(4) <http://dbpedia.org/resource/Bob_Marley> <http://www.w3.org/2000/01/rdf-schema#seeAlso> <http://dbpedia.org/resource/Rastafari> .
(5) <http://dbpedia.org/resource/Bob_Marley> <http://dbpedia.org/ontology/birthPlace> <http://dbpedia.org/resource/Jamaica> .
(6) <http://dbpedia.org/resource/Jamaica> <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> <http://schema.org/Country> .
(7) <http://dbpedia.org/resource/Jamaica> <http://www.w3.org/2000/01/rdf-schema#label> "Jamaica"@en .
(8) <http://dbpedia.org/resource/Jamaica> <http://www.w3.org/2000/01/rdf-schema#label> "Giamaica"@it .
(9) <http://dbpedia.org/resource/Jamaica> <http://www.w3.org/2003/01/geo/wgs84_pos#lat> "17.9833"^^<http://www.w3.org/2001/XMLSchema#float> .
(10) <http://dbpedia.org/resource/Jamaica> <http://www.w3.org/2003/01/geo/wgs84_pos#long> "-76.8"^^<http://www.w3.org/2001/XMLSchema#float> .
(11) <http://dbpedia.org/resource/Jamaica> <http://xmlns.com/foaf/0.1/homepage> <http://jis.gov.jm/> .
```

Entity Predicate Literal Literal tag

we can see the foaf ontology used once more to connect the entity of Jamaica to its homepage URL (subset of URI).

Turtle

- you can define prefixes at the beginning of the .ttl file.

```
@prefix dbr: <http://dbpedia.org/resource/> .
@prefix dbo: <http://dbpedia.org/ontology/> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix foaf: <http://xmlns.com/foaf/0.1/> .
@prefix geo: <http://www.w3.org/2003/01/geo/wgs84_pos#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
@prefix schema: <http://schema.org/> .
```

the long repeated sections of the URIs can be written in their short form.

This allows the subject `<http://xmlns.com/foaf/0.1/Person>` to be shortened to `foaf:Person`.

```
dbr:Bob_Marley
  a foaf:Person ;
  rdfs:label "Bob Marley"@en ;
  rdfs:label "Bob Marley"@fr ;
  rdfs:seeAlso dbr:Rastafari ;
  dbo:birthPlace dbr:Jamaica .
```

One unique shortening is the predicate `<http://www.w3.org/1999/02/22-rdf-syntax-ns#type>` which is so common, it is represented simply with the letter `a`

```
<http://dbpedia.org/resource/Bob_Marley>
<http://www.w3.org/1999/02/22-rdf-syntax-ns#type>
<http://xmlns.com/foaf/0.1/Person> .
```

can be shortened
in Turtle to

```
dbr:Bob_Marley a foaf:Person
```

```
dbr:Jamaica
  a schema:Country ;
  rdfs:label "Jamaica"@en ;
  rdfs:label "Giamaica"@it ;
  geo:lat "17.9833"^^xsd:float ;
  geo:long "-76.8"^^xsd:float ;
  foaf:homepage <http://jis.gov.jm/> .
```

data about Bob Marley and Jamaica is separated into separate blocks. This grouping along with defined prefixes, makes Turtle format a lot easier to understand than N-Triples. Related information to the subject is separated with a semi-colon and finished with a full-stop and a new line to indicate a new subject.

RDF/XML

```
<?xml version="1.0" encoding="utf-8" ?>
<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
  xmlns:foaf="http://xmlns.com/foaf/0.1/"
  xmlns:rdfs="http://www.w3.org/2000/01/rdf-schema#"
  xmlns:ns0="http://dbpedia.org/ontology/"
  xmlns:schema="http://schema.org/"
  xmlns:geo="http://www.w3.org/2003/01/geo/wgs84_pos#">
```

```
<foaf:Person rdf:about="http://dbpedia.org/resource/Bob_Marley">
  <rdfs:label xml:lang="en">Bob Marley</rdfs:label>
  <rdfs:label xml:lang="fr">Bob Marley</rdfs:label>
  <rdfs:seeAlso rdf:resource="http://dbpedia.org/resource/Rastafari"/>
  <ns0:birthPlace>
    <schema:Country rdf:about="http://dbpedia.org/resource/Jamaica">
      <rdfs:label xml:lang="en">Jamaica</rdfs:label>
      <rdfs:label xml:lang="it">Giamaica</rdfs:label>
      <geo:lat rdf:datatype="http://www.w3.org/2001/XMLSchema#float">17.9833</geo:lat>
      <geo:long rdf:datatype="http://www.w3.org/2001/XMLSchema#float">-76.8</geo:long>
      <foaf:homepage rdf:resource="http://jis.gov.jm/" />
    </schema:Country>
  </ns0:birthPlace>

</foaf:Person>

</rdf:RDF>
```

prefixes can be defined at the top of RDF/XML files to avoid unnecessary repetition of URIs.

RDF format is considered the standard format.

What is SPARQL?

- SPARQL (SPARQL Protocol and RDF Query Language) is the query language
- The following query returns every triple

```
SELECT ?subject ?predicate ?object
WHERE {
  ?subject ?predicate ?object .
}
```

The ? indicates a variable so ?example is a variable called “example”

```
SELECT *
WHERE {
  ?s ?p ?o .
}
LIMIT 200
```

this query only returns 200 triples.

```
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>

SELECT *
WHERE {
    ?entity rdfs:label ?name .
}
LIMIT 20
```

Entities are connected to their human-readable names with the predicate `rdfs:label` so `?entity rdfs:label ?name` matches any triple with the predicate `rdfs:label` and returns `?entity` and `?name`.

Olympics RDF data Examples:

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <rdf:RDF
3   xmlns:dbo="http://dbpedia.org/ontology/"
4   xmlns:dbp="http://dbpedia.org/property/"
5   xmlns:walls="http://wallscope.co.uk/ontology/olympics/"
6   xmlns:skos="http://www.w3.org/2004/02/skos/core#"
7   xmlns:rdfs="http://www.w3.org/2000/01/rdf-schema#"
8   xmlns:foaf="http://xmlns.com/foaf/0.1/"
9   xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#">
10
11 <rdf:Description rdf:about="http://wallscope.co.uk/resource/olympics/athlete/ADijiang">
12   <rdf:type rdf:resource="http://xmlns.com/foaf/0.1/Person"/>
13 </rdf:Description>
14
15 <rdf:Description rdf:about="http://wallscope.co.uk/resource/olympics/gender/M">
16   <rdf:type rdf:resource="http://www.w3.org/2004/02/skos/core#Concept"/>
17   <rdfs:label xml:lang="en">male</rdfs:label>
18 </rdf:Description>
19
20 <rdf:Description rdf:about="http://wallscope.co.uk/resource/olympics/athlete/ADijiang">
21   <foaf:gender rdf:resource="http://wallscope.co.uk/resource/olympics/gender/M"/>
22   <foaf:age rdf:datatype="http://www.w3.org/2001/XMLSchema#int">24</foaf:age>
23   <dbo:height rdf:datatype="http://www.w3.org/2001/XMLSchema#int">180</dbo:height>
24   <dbo:weight rdf:datatype="http://www.w3.org/2001/XMLSchema#double">80</dbo:weight>
25 </rdf:Description>
26
27 <rdf:Description rdf:about="http://wallscope.co.uk/resource/olympics/team/China">
28   <rdf:type rdf:resource="http://dbpedia.org/ontology/SportsTeam"/>
29   <rdfs:label xml:lang="en">China</rdfs:label>
30 </rdf:Description>
31
```

Query

- Lists the names of every gold medallist (excluding duplicates):

```
PREFIX walls: <http://wallscope.co.uk/ontology/olympics/>  
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
```

listing the prefixes used in the query to make it more readable.

```
SELECT DISTINCT ?name  
WHERE {  
  ?instance walls:athlete ?athlete ;  
  walls:medal <http://wallscope.co.uk/resource/olympics/medal/Gold> .  
  ?athlete rdfs:label ?name .  
}
```

selected to return the variable called ?name

DISTINCT clause after SELECT filters the names to remove duplicates.

athletes are linked to their medals through the instance entity

```
<rdf:Description rdf:about="http://wallscope.co.uk/resource/olympics/instance/TugOfWarMensTugOfWar/1900Summer/EdgarLindenauAabye">  
  <walls:athlete rdf:resource="http://wallscope.co.uk/resource/olympics/athlete/EdgarLindenauAabye"/>  
  <walls:games rdf:resource="http://wallscope.co.uk/resource/olympics/games/1900/Summer"/>  
  <walls:event rdf:resource="http://wallscope.co.uk/resource/olympics/event/TugOfWarMensTugOfWar"/>  
  <walls:medal rdf:resource="http://wallscope.co.uk/resource/olympics/medal/Gold"/>  
</rdf:Description>
```

The first triple `?instance walls:athlete ?athlete ;` grabs every entity with an outgoing predicate `walls:athlete` and the athlete entity.

`walls:medal <http://wallscope.co.uk/resource/olympics/medal/Gold> .` connects the instance entities to their Gold medal entities.

`?athlete rdfs:label ?name .`

we get their labels and assign them the variable `?name` as this is what we want to return.

- a list of human-readable names of athletes that got gold medals at the Olympics (with duplicates removed) as desired:

	name
1	"Edgar Lindenau Aabye"@en
2	"Paavo Johannes Aaltonen"@en
3	"Kjetil Andr Aamodt"@en
4	"Ragnhild Margrethe Aamodt"@en
5	"Roald Edgar Aas"@en
6	"Thomas Valentin Aas"@en
7	"Ren Abadie"@en
8	"Aleksey Aleksandrovich Abalmasov"@en
9	"Luc Abalo"@en

Quary

- lists each country alongside the average height and weight of its athletes (sorted by the average height):

```
PREFIX walls: <http://wallscope.co.uk/ontology/olympics/>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX dbo: <http://dbpedia.org/ontology/>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>

SELECT ?countryCode
      (AVG(?height) As ?avgHeight)
      (AVG(?weight) As ?avgWeight)
WHERE {
  ?noc dbo:ground ?team ;
      rdfs:label ?countryCode .

  ?athlete rdf:type    foaf:Person ;
           dbo:team    ?team ;
           dbo:height  ?height ;
           dbo:weight  ?weight .
}
GROUP BY ?countryCode
ORDER BY DESC(?avgHeight)
```

- All SPARQL prefixes
- <https://prefix.cc/popular/all.sparql>

Tools

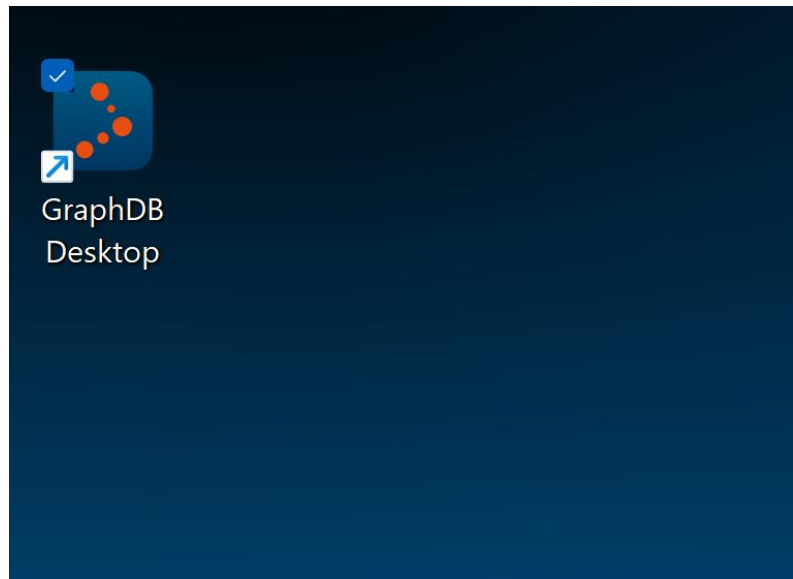
Elham

GraphDB

[Download here:](#)

<https://graphdb.ontotext.com/>

 GraphDB_Desktop-10.6.2.msi	17/03/2024 19:23	Windows Installer ...	310.544 KB
--	------------------	-----------------------	------------



GraphDB

- <http://localhost:7200/>.

The screenshot displays the GraphDB web interface. On the left is a dark blue sidebar with the GraphDB logo and a list of navigation options: Import, Explore, SPARQL, Monitor, Setup, Lab, and Help. Each option has a corresponding icon and a dropdown arrow. The main content area is white and features a 'Welcome to GraphDB' modal dialog. The dialog contains a search icon, a repository selector set to 'my_repo', and a language selector set to 'en'. The welcome message states: 'Welcome to GraphDB. GraphDB is a graph database compliant with RDF and SPARQL specifications. It supports open APIs based on the RDF4J project and enables fast publishing of linked data on the web. The Workbench is used for searching, exploring and managing GraphDB repositories. Run our interactive guides for an easy way into learning some basic concepts and getting started with GraphDB. You can always come back to this panel by pressing the GraphDB icon in the top left corner.' At the bottom of the dialog are two buttons: 'Hide panel' and 'Take me to the guides'. A pagination bar at the bottom right shows five numbered tabs, with the first tab (1) being active.

Creation of repository

The screenshot displays the SPARQL interface's 'Repositories' management page. The left sidebar features a navigation menu with the following items: SPARQL, Monitor, Setup, **Repositories** (highlighted), Users and Access, ACL Management, My Settings, Connectors, Cluster, Plugins, Namespaces, and Autocomplete. The main content area is titled 'Repositories' and indicates 'Repositories from: Local'. A table lists the existing repository:

Repository Name	Status
my_repo	RUNNING

Below the table, there are two buttons for repository management: 'Create new repository' and 'Attach remote location'.

Import data

GraphDB

Import

Explore

SPARQL

Monitor

Setup

Lab

Help

Import

User dataServer files

Upload RDF files
All RDF formats, up to 200 MB

Get RDF data from a URL
All RDF formats

Import RDF text snippet
Type or paste RDF data

☐

data-example.rdf

Imported successfully in less than a second.



Import



my_repo

en

Help



GraphDB 10.6.2 • RDF4J 4.3.9 • Connectors 16.2.6 • Workbench 2.6.2 • © 2002–2024 Ontotext AD. All rights reserved.



Import

Explore

Graphs overview

Class hierarchy

Class relationships

Visual graph

Similarity

SPARQL

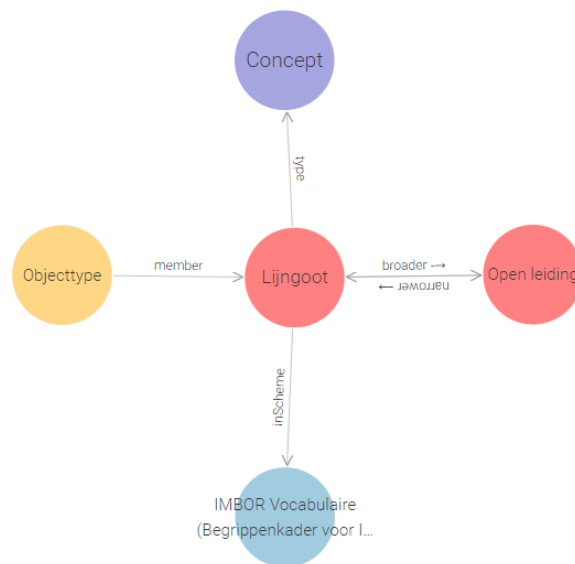
Monitor

Setup

Visual graph i

my_repo

en





Import

Explore

SPARQL

Monitor

Setup

Lab

Help

SPARQL Query & Update



my_repo

en

Editor only

Editor and results

Results only



Unnamed

SPARQL Select template



```
1 SELECT ?s ?p ?o
2 WHERE {
3   ?s ?p ?o .
4 } LIMIT 10
```



Run

keyboard shortcuts

Table

Raw response

Pivot Table

Google Chart

Download as

Blazegraph (Udemy course recommended this one)

<https://blazegraph.com/> (to download and install it)

Run the command (java -server -Xmx4g -jar blazegraph.jar)
(sometimes it conflicts with vpn and gives the error on that.)

- https://github.com/blazegraph/database/wiki/Quick_Start
- If we want to work from a local machine just use this:
- <http://localhost:9999/blazegraph/>
- Portnumber: 9999

```

at com.bigdata.rdf.sail.webapp.StandaloneNanoSparqlServer.main(StandaloneNanoSparqlServer.java:191)
WARN : MapgraphServletProxy.java:67: Running without GPU Acceleration. See https://www.blazegraph.com/product/gpu-acce
lerated/.
WARN : MapgraphServletProxy.java:67: Running without GPU Acceleration. See https://www.blazegraph.com/product/gpu-acce
lerated/.
PS C:\RDFFlow> java -server -Xmx4g -jar blazegraph.jar
INFO: com.bigdata.util.config.LogUtil: Configure: jar:file:/C:/RDFFlow/blazegraph.jar!/log4j.properties
WARN : SystemUtil.java:71: No CUParser for this platform - looking for class: [org.apache.system.Windows11]

```

BlazeGraph(TM) Graph Engine

Flexible
Reliable
Affordable
Web-Scale Computing for the Enterprise

Copyright SYSTAP, LLC DBA Blazegraph 2006-2016. All rights reserved.

```

VC0-8324FY3.internal.visma.com
Fri Mar 22 12:50:28 CET 2024
Windows 11/10.0 amd64
amd64 Family n, Model n, Stepping n, Undeterminable #CPU=1
Amazon.com Inc. 11.0.19
freeMemory=516947968
buildVersion=2.1.6-SNAPSHOT
gitCommit=6b0c935523f5064b80279b30a5175a858cddd2a1

```

Dependency	License
ICU	http://source.icu-project.org/repos/icu/icu/t
bigdata-ganglia	http://www.apache.org/licenses/LICENSE-2.0.html

httpcore	http://www.apache.org/licenses/LICENSE-2.0.html
httpmime	http://www.apache.org/licenses/LICENSE-2.0.html
jackson-core	http://www.apache.org/licenses/LICENSE-2.0.html
jetty	http://www.apache.org/licenses/LICENSE-2.0.html
jquery	https://github.com/jquery/jquery/blob/master/MIT-LICENSE.txt
jsonld	https://raw.githubusercontent.com/jsonld-java/jsonld-java/master/LICENSE
log4j	http://www.apache.org/licenses/LICENSE-2.0.html
lucene	http://www.apache.org/licenses/LICENSE-2.0.html
nanohhttp	http://elonen.iki.fi/code/nanohhttpd/#license
rexster-core	https://github.com/tinkerpop/rexster/blob/master/LICENSE.txt
river	http://www.apache.org/licenses/LICENSE-2.0.html
semargl	https://github.com/levkhomich/semargl/blob/master/LICENSE
servlet-api	http://www.apache.org/licenses/LICENSE-2.0.html
sesame	http://www.openrdf.org/download.jsp
slf4j	http://www.slf4j.org/license.html
zookeeper	http://www.apache.org/licenses/LICENSE-2.0.html

```

WARN : NanoSparqlServer.java:517: Starting NSS
WARN : ServiceProviderHook.java:171: Running.
serviceURL: http://127.0.0.1:9999

```

Welcome to the BlazeGraph(TM) Database.

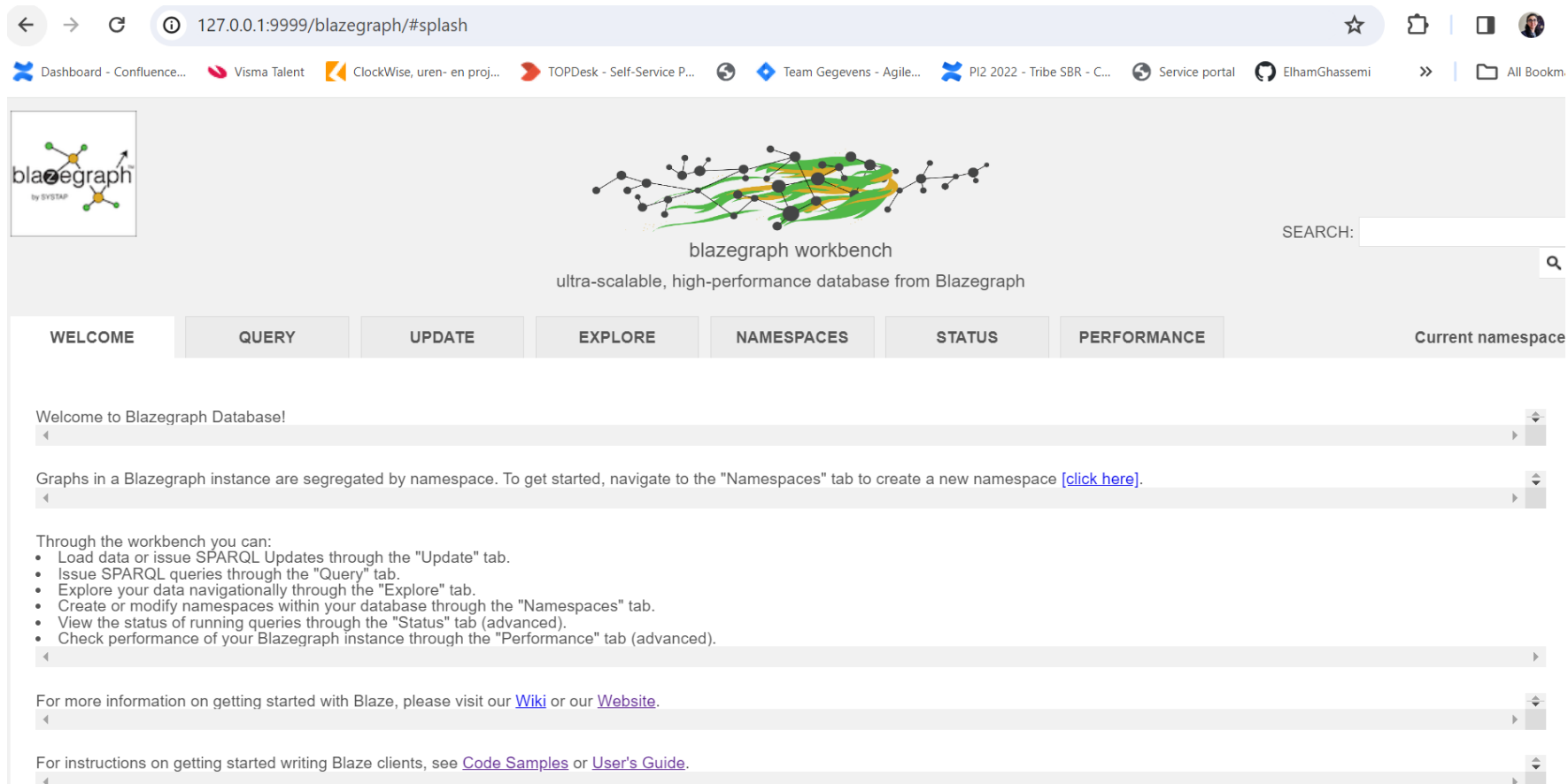
Go to <http://127.0.0.1:9999/blazegraph/> to get started.

```

WARN : MapgraphServletProxy.java:67: Running without GPU Acceleration. See https://www.blazegraph.com/product/gpu-acce
lerated/.
WARN : MapgraphServletProxy.java:67: Running without GPU Acceleration. See https://www.blazegraph.com/product/gpu-acce
lerated/.

```

Copy this in your
browser to start it



Quick start with Blazegraph

- First create a new database where we are going to be uploading our dataset. We go to namespace and create our demo in quad mode.

WELCOME

QUERY

UPDATE

EXPLORE

NAMESPACES

STATUS

PERFORMANCE

Current namespace: demo

Namespaces

demo	Use	Delete	Properties	Rebuild Full Text Index	Clone	Service Description
kb	In use	Delete	Properties	Rebuild Full Text Index	Clone	Service Description

[Download VoID description of all namespaces](#)

demo

Create namespace

There are a number of features to enable. There's full documentation [here](#). You must select "Use" after you have created the KB.

A quick reference is below:

o PropertyGraph: Select triples.

o RDF + SPARQL with named graphs: Select quads mode.

o Support for [Reification Done Right \(RDR\)](#). Select rdr mode.

Name: Mode: triples Inference: ☐ Isolatable indices: ☐ Full text index: ☐ Enable geospatial: ☐

Create namespace

Load data

- Use LOAD operation: (it is beneficial when we have multiple files to upload them)

The screenshot shows a web-based SPARQL query editor. At the top, there are tabs for 'Namespace browser', 'Query', 'View', 'Query logs', 'Query history', 'Query', and 'Log'. The 'Query' tab is active, displaying a SPARQL query in a text area: `1 LOAD <file:/C:/RDFFlow/demo/General_Country_Info_Dataset.ttl> INTO GRAPH <http://example.com/demo/General_Country_Info_Graph>`. Below the query area, there is a 'Choose File' button and the text 'No file chosen'. To the right, there are dropdown menus for 'Type: SPARQL Update' and 'Format: N-Quads'. A link for 'Advanced features' is also present. A horizontal progress bar is shown below the query area, with an 'Update' button centered underneath it. At the bottom of the interface, a status bar displays performance metrics: 'totalElapsed=344ms, elapsed=344ms, connFlush=0ms, batchResolve=0, whereClause=0ms, deleteClause=0ms, insertClause=0ms'. A 'Grammarly' watermark is visible at the bottom center.

```
1 LOAD <file:/C:/RDFFlow/demo/General_Country_Info_Dataset.ttl> INTO GRAPH <http://example.com/demo/General_Country_Info_Graph>
```

Choose File No file chosen

Type: SPARQL Update Format: N-Quads

[Advanced features](#)

Update

totalElapsed=344ms, elapsed=344ms, connFlush=0ms, batchResolve=0, whereClause=0ms, deleteClause=0ms, insertClause=0ms

Grammarly

Specify the prefix for our graph

- INSERT DATA operation containing statements involving the named graph: General_Country_Info_Graph

[Wiki](#) - [SPARQL Update](#)

Namespace shortcuts:

```
1 INSERT DATA
2 {
3   @prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
4   @prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
5   @prefix dcterms: <http://purl.org/dc/terms/> .
6   @prefix : <http://example.com/demo/> .
7
8   :General_Country_Info_Graph dcterms:title "General country information graph" ;
9     dcterms:description "Named graph containing basic physical and human geography characteristics of countries" ;
10    dcterms:source <https://www.geonames.org/> .
11 }
12
13
```

No file chosen

Type: Format:

[Advanced features](#)

Quary

- Describe the named graph

```
1 PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
2 PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
3 PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
4 PREFIX skos: <http://www.w3.org/2004/02/skos/core#>
5 PREFIX dcterms: <http://purl.org/dc/terms/>
6 PREFIX : <http://example.com/demo/>
7
8 DESCRIBE :General_Country_Info_Graph
```

[Advanced features](#)

Execute

Clear

subject	predicate	object
http://example.com/demo/General_Country_Info_Graph	dcterms:source	https://www.geonames.org/
http://example.com/demo/General_Country_Info_Graph	dcterms:description	Named graph containing basic physical and human geography characteristics of countries
http://example.com/demo/General_Country_Info_Graph	dcterms:title	General country information graph

Quary

- Exposing the full vocabulary for the graph

[Wiki](#) - [SPARQL Query](#)

```
1 PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
2 PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
3 PREFIX skos: <http://www.w3.org/2004/02/skos/core#>
4 PREFIX : <http://example.com/demo/>
5
6 SELECT DISTINCT ?t ?p ?pType
7 WHERE {
8     ?s ?p ?o ;
9     a ?t .
10
11     BIND (
12     COALESCE(
13         IF(isIRI(?o), "Resource Predicate", 1/0),
14         "Literal Predicate"
15     ) AS ?pType
16 )
17 }
18 GROUP BY ?t ?p ?pType
19 ORDER BY ASC(?t) ASC(?p)
```

t	p	pType
http://example.com/demo/Continent	http://example.com/demo/continent_code	Literal Predicate
http://example.com/demo/Continent	http://example.com/demo/continent_name	Literal Predicate
http://example.com/demo/Continent	rdf:type	Resource Predicate
http://example.com/demo/Continent	skos:prefLabel	Literal Predicate
http://example.com/demo/Country	http://example.com/demo/area_in_sq_km	Literal Predicate
http://example.com/demo/Country	http://example.com/demo/capital	Literal Predicate
http://example.com/demo/Country	http://example.com/demo/country_name	Literal Predicate
http://example.com/demo/Country	http://example.com/demo/has_country_neighbour	Resource Predicate
http://example.com/demo/Country	http://example.com/demo/iso_3166-1_alpha-2_code	Literal Predicate
http://example.com/demo/Country	http://example.com/demo/iso_3166-1_alpha-3_code	Literal Predicate
http://example.com/demo/Country	http://example.com/demo/iso_3166-1_numeric_code	Literal Predicate
http://example.com/demo/Country	http://example.com/demo/part_of_continent	Resource Predicate
http://example.com/demo/Country	http://example.com/demo/population	Literal Predicate
http://example.com/demo/Country	http://example.com/demo/uses_currency	Resource Predicate
http://example.com/demo/Country	rdf:type	Resource Predicate
http://example.com/demo/Country	skos:prefLabel	Literal Predicate
http://example.com/demo/Country_Language	http://example.com/demo/language_value	Resource Predicate
http://example.com/demo/Country_Language	http://example.com/demo/language_variant	Literal Predicate
http://example.com/demo/Country_Language	http://example.com/demo/rank	Literal Predicate
http://example.com/demo/Country_Language	http://example.com/demo/spoken_in	Resource Predicate
http://example.com/demo/Country_Language	rdf:type	Resource Predicate
http://example.com/demo/Country_Neighbour	http://example.com/demo/country_neighbour_value	Resource Predicate
http://example.com/demo/Country_Neighbour	rdf:type	Resource Predicate
http://example.com/demo/Currency	http://example.com/demo/currency_code	Literal Predicate

Sum in the query

```
1 PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
2 PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
3 PREFIX skos: <http://www.w3.org/2004/02/skos/core#>
4 PREFIX dcterms: <http://purl.org/dc/terms/>
5 PREFIX : <http://example.com/demo/>
6
7 ### what is the total land mass area (to the nearest sq km) of given countinen, e.g., Asia?
8 SELECT (sum(?cArea) As ?ctArea)
9 WHERE {
10   ?c :part_of_continent ?ct;
11     :area_in_sq_km ?cArea.
12   ?ct :continent_name "Asia".
13 }
14
15 }
16 GROUP BY ?ct
17
```

[Advanced features](#)

Execute

Clear

ctArea
3.16030937E7

Sum: Aggregate function to calculate the sum of numbers.

Round : rounds a number to the nearest integer.

CEIL: rounds up a number to the nearest integer.

FLOOR: rounds down a number to the nearest integer.

RAND: returns a random value between 0 and 1.

AVG: Aggregate function to calculate the average value from a list of numbers.

MIN,MAX: Aggregate functions to work out the minimum and maximum values, respectively. Numbers are sorted numerically.

ABS: returns the absolute value of a number.

IN and NOT IN filter

```
1 PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
2 PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
3 PREFIX skos: <http://www.w3.org/2004/02/skos/core#>
4 PREFIX dcterms: <http://purl.org/dc/terms/>
5 PREFIX : <http://example.com/demo/>
6
7 ### what is the total population of each continent excluding Antarctica oceania?
8 SELECT ?ctName(sum(?cPop) As ?ctPop)
9 WHERE {
10   ?c :part_of_continent ?ct;
11     :population ?cPop.
12   ?ct :continent_name ?ctName
13 }
14
15
16 }
17 GROUP BY ?ctName
18
```

[Advanced features](#)

Execute Clear

ctName	ctPop
Asia	4522874728
Africa	1268766150
Antarctica	170
South America	423597139
Oceania	43093797
North America	583536773
Europe	758597538

We can see Antarctica and Oceania is also in the results and we want to exclude them.

IN and NOT IN filter

```
1 PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
2 PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
3 PREFIX skos: <http://www.w3.org/2004/02/skos/core#>
4 PREFIX dcterms: <http://purl.org/dc/terms/>
5 PREFIX : <http://example.com/demo/>
6
7 ### what is the total population of each continent excluding Antarctica and oceania?
8 SELECT ?ctName(sum(?cPop) As ?ctPop)
9 WHERE {
10   ?c :part_of_continent ?ct;
11   :population ?cPop.
12   ?ct :continent_name ?ctName
13
14   FILTER(?ctName != "Antarctica" && ?ctName != "Oceania")
15
16 }
17 GROUP BY ?ctName
18
19
```

[Advanced features](#)

Execute Clear

ctName	ctPop
South America	423597139
North America	583536773
Asia	4522874728
Africa	1268766150
Europe	758597538

We can filter two
of continents

IN and NOT IN filter

```
1 PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
2 PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
3 PREFIX skos: <http://www.w3.org/2004/02/skos/core#>
4 PREFIX dcterms: <http://purl.org/dc/terms/>
5 PREFIX : <http://example.com/demo/>
6
7 ### what is the total population of each continent excluding Antarctica and oceania?
8 SELECT ?ctName(sum(?cPop) As ?ctPop)
9 WHERE {
10     #VALUES ?inc {"Antarctica" "Oceania"}
11     ?c :part_of_continent ?ct;
12         :population ?cPop.
13     ?ct :continent_name ?ctName
14
15     #FILTER(?ctName = ?exc)
16     FILTER(?ctName NOT IN ("Antarctica" , "Oceania"))
17
18 }
19
20 GROUP BY ?ctName
21
```

[Advanced features](#)

Execute

Clear

ctName	ctPop
Asia	4522874728
Africa	1268766150
South America	423597139
North America	583536773
Europe	758597538

Challenge

Write a SPARQL query to find out the population of each of the countries of Central America.

Central America consists of countries with the following names: "Belize", "Costa Rica", "El Salvador", "Guatemala", "Honduras", "Nicaragua" and "Panama".

In your query, use the functions `FILTER` and `IN` and order the results in descending order of population.

```
1 PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
2 PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
3 PREFIX skos: <http://www.w3.org/2004/02/skos/core#>
4 PREFIX dct: <http://purl.org/dc/terms/>
5 PREFIX : <http://example.com/demo/>
6
7 SELECT ?cName ?cPop
8 WHERE {
9   ?c :population ?cPop ;
10      :country_name ?cName .
11
12   FILTER(?cName IN("Belize", "Costa Rica", "El Salvador", "Guatemala", "Honduras", "Nicaragua", "Panama"))
13 }
14 ORDER BY DESC(?cPop)
15
```

[Advanced features](#)

Execute Clear

cName	cPop
Guatemala	17247807
Honduras	9587522
Nicaragua	6465513
El Salvador	6420744
Costa Rica	4999441
Panama	4176873
Belize	383071

Function STRENDS, STRSTARTS

```
1 PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
2 PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
3 PREFIX skos: <http://www.w3.org/2004/02/skos/core#>
4 PREFIX dcterms: <http://purl.org/dc/terms/>
5 PREFIX : <http://example.com/demo/>
6
7 ### List the country names of countries in the continent "Asia" whose names end in 'stan'.
8 #For these countries, list the names of their associated languages and the rank of the languages.
9 #Order the results in ascending order of country name and language rank.
10
11
12 SELECT ?cName ?langN ?r
13 WHERE {
14     ?c :country_name ?cName ;
15         :part_of_continent/:continent_name "Asia" .
16
17     ?clang :spoken_in ?c ;
18         :language_value/:language_name ?langN ;
19         :rank ?r .
20
21     FILTER(STRENDS(?cName, "stan"))
22 }
23 ORDER BY ASC(?cName) ASC(?r)
```

cName	langN	r
Afghanistan	Persian	1
Afghanistan	Pushto	2
Afghanistan	Uzbek	3
Kazakhstan	Kazakh	1
Kazakhstan	Russian	2
Kyrgyzstan	Kirghiz	1
Kyrgyzstan	Uzbek	2
Kyrgyzstan	Russian	3
Pakistan	Urdu	1
Pakistan	English	2
Pakistan	Panjabi	3
Tajikistan	Tajik	1
Tajikistan	Russian	2
Turkmenistan	Turkmen	1
Turkmenistan	Russian	2
Turkmenistan	Uzbek	3
Uzbekistan	Uzbek	1
Uzbekistan	Russian	2
Uzbekistan	Tajik	3



Functions

STRBEFORE(string, compare_string) returns the part of the string before the compare string.

Example: **STRBEFORE("South Africa", "th")** returns **Sou**. If the compare string is not found it returns empty.

STRAFTER

Purpose: **STRAFTER(string, compare_string)** returns the part of the string after the compare string.

Example: **STRAFTER("Thailand", "ai")** returns **land**. If the compare string is not found it returns empty.

STRLEN

Purpose: **STRLEN(string)** returns the length of a string.

Example: **STRLEN(?country)** where **?country** is **China** will return **5**.

CONTAINS

Purpose: **CONTAINS(string, compare_string)** checks if the string contains the compare string.

Example: **CONTAINS("Madagascar", "dag")** returns true, meaning the string contains **dag**.

SUBSTR

Purpose: **SUBSTR(string, start_position, string_length)** returns a substring of a string starting at the position marked by **start_position**, and will have the given length supplied through **string_length**.

Example: **SUBSTR(?country, 7, 4)** where **?country** is **Costa Rica** will return **Rica**. Omitting the length as in **SUBSTR(?country, 7)** returns **Rica**, i.e. the remaining string from the starting position.

UCASE

Purpose: **UCASE(string)** returns the string in upper case.

Example: **UCASE("Japan")** will be returned as **JAPAN**.

LCASE

Purpose: **LCASE(string)** returns the string in lower case.

Example: **LCASE("UK")** will be returned as **uk**.

CONCAT

Purpose: **CONCAT(string1, string2, ...)** returns the concatenation of two or more strings.

Example: **CONCAT("Republic", " of ", "Ireland")** returns Republic of Ireland.

ENCODE_FOR_URI

Purpose: **ENCODE_FOR_URI(string)** converts any special characters in the string for use in a URL for the web.

Example: **ENCODE_FOR_URI("Washington DC")** returns **Washington%20DC**.

LANGMATCHES

Purpose: **LANGMATCHES(language_tag, language_range)** checks if the language tag matches the language range. See [langMatches](#) for more info.

Example: **FILTER LANGMATCHES(LANG(?label), "en")** filters language tags in the English language. The language tags may contain region as well, e.g. **en-IE**. A language range of ***** matches any non-empty language-tag string.

REGEX

Purpose: **REGEX(string, pattern, flag)** checks if the string matches the pattern. The pattern may contain different special characters. See [regex](#) for more info.

REPLACE

Purpose: **REPLACE(string, pattern, replacement, flag)** returns the string after replacing all occurrences of **pattern** in **string** with **replacement**. **pattern** is interpreted the same as in REGEX. **flag** is optional.

Example: **REPLACE("United_Kingdom", "[_]", " ")** removes all the underscores from the original string and replaces them with spaces.

Transposing data in tables with OPTIONAL

For the countries in "Asia" whose names end in 'stan', list the names of their associated languages as data transposed in such a way that the result table displays, for each country, its rank 1 language as primary language in one column, rank 2 language as secondary language in another column, and rank 3 language as tertiary language in another column.

```
SELECT ?cName ?primaryLang ?secondaryLang ?tertiaryLang
WHERE {
  ?c :country_name ?cName ;
     :part_of_continent/:continent_name "Asia" .

  OPTIONAL {
    ?clang1 :spoken_in ?c ;
            :language_value/:language_name ?primaryLang ;
            :rank 1 .
  }
  OPTIONAL {
    ?clang2 :spoken_in ?c ;
            :language_value/:language_name ?secondaryLang ;
            :rank 2 .
  }
  OPTIONAL {
    ?clang3 :spoken_in ?c ;
            :language_value/:language_name ?tertiaryLang ;
            :rank 3 .
  }

  FILTER(STRENDS(?cName, "stan"))
}
GROUP BY ?cName ?primaryLang ?secondaryLang ?tertiaryLang
ORDER BY ASC(?cName)
```

cName	primaryLang	secondaryLang	tertiaryLang
Afghanistan	Persian	Pushto	Uzbek
Kazakhstan	Kazakh	Russian	
Kyrgyzstan	Kirghiz	Uzbek	Russian
Pakistan	Urdu	English	Panjabi
Tajikistan	Tajik	Russian	
Turkmenistan	Turkmen	Russian	Uzbek
Uzbekistan	Uzbek	Russian	Tajik

Subquery

- List the names of the 5 most populated countries in the continent "Europe" and the names of their corresponding neighbours. Order the results in ascending order of the names of these 5 countries.

```
1 PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
2 PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
3 PREFIX skos: <http://www.w3.org/2004/02/skos/core#>
4 PREFIX dcterms: <http://purl.org/dc/terms/>
5 PREFIX : <http://example.com/demo/>
6 SELECT ?cName ?cPop
7 WHERE {
8   ?c :population ?cPop;
9     :part_of_continent/:continent_name "Europe" .
10   ?c :country_name ?cName.
11 }
12 }
13
14
15 ORDER BY DESC(?cPop)
16 LIMIT 5
```

[Advanced features](#)

This query pulls the top 5 most populated countries in Europe.

cName	cPop
Russia	144478050
Germany	82927922
France	66987244
United Kingdom	66488991
Italy	60431283

Subquery

```
1 PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
2 PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
3 PREFIX skos: <http://www.w3.org/2004/02/skos/core#>
4 PREFIX dcterms: <http://purl.org/dc/terms/>
5 PREFIX : <http://example.com/demo/>
6 SELECT ?cName1 ?cName2 ?cPop
7 WHERE {
8   ?c :population ?cPop;
9       :part_of_continent/:continent_name "Europe" .
10   ?c :country_name ?cName1;
11       :has_country_neighbour/:country_neighbour_value/:country_name ?cName2.
12 }
13
14
15
16 ORDER BY DESC(?cPop)
17 LIMIT 5
```

[Advanced features](#)

Exec

cName1	cName2	cPop
Russia	Azerbaijan	144478050
Russia	Belarus	144478050
Russia	China	144478050
Russia	Estonia	144478050
Russia	Finland	144478050

This query gives the name of the neighboring countries but it is not correct for the request.

Subquery

```
1 PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
2 PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
3 PREFIX skos: <http://www.w3.org/2004/02/skos/core#>
4 PREFIX dcterms: <http://purl.org/dc/terms/>
5 PREFIX : <http://example.com/demo/>
6
7 SELECT ?cName1 ?cName2 ?cPop
8 WHERE {
9     ?c :country_name ?cName1;
10        :has_country_neighbour/:country_neighbour_value/:country_name ?cName2.
11     { SELECT ?c
12       WHERE {
13         ?c :population ?cPop;
14            :part_of_continent/:continent_name "Europe" .
15       }
16     }
17     ORDER BY DESC(?cPop)
18     LIMIT 5
19 }
20 }
21
22 ORDER BY ASC(?cName1)
```

cName1	cName2	cPop
France	Andorra	
France	Belgium	
France	Switzerland	
France	Germany	
France	Spain	
France	Italy	
France	Luxembourg	
France	Monaco	
Germany	Austria	
Germany	Belgium	
Germany	Switzerland	
Germany	Czechia	
Germany	Denmark	
Germany	France	
Germany	Luxembourg	
Germany	Netherlands	
Germany	Poland	
Italy	Austria	
Italy	Switzerland	
Italy	France	
Italy	Slovenia	
Italy	San Marino	
Italy	Vatican	
Russia	Azerbaijan	
Russia	Belarus	
Russia	China	
Russia	Estonia	
Russia	Finland	
Russia	Georgia	
Russia	North Korea	
Russia	Kazakhstan	
Russia	Lithuania	
Russia	Latvia	
Russia	Mongolia	
Russia	Norway	
Russia	Poland	
Russia	Ukraine	
United Kingdom	Ireland	

Concatenating elements with GROUP_CONCAT

```
1 PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
2 PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
3 PREFIX skos: <http://www.w3.org/2004/02/skos/core#>
4 PREFIX dcterms: <http://purl.org/dc/terms/>
5 PREFIX : <http://example.com/demo/>
6
7 SELECT ?cName1 (GROUP_CONCAT(?cName2;SEPARATOR=",") AS ?neighbours)
8 WHERE {
9   ?c :country_name ?cName1;
10      :has_country_neighbour/:country_neighbour_value/:country_name ?cName2.
11   { SELECT ?c
12     WHERE {
13       ?c :population ?cPop;
14          :part_of_continent/:continent_name "Europe" .
15     }
16   }
17   ORDER BY DESC(?cPop)
18   LIMIT 5
19 }
20 }
21 GROUP BY ?cName1
22 ORDER BY ASC(?cName1)
```

[Advanced features](#)

Execute Clear

cName1	neighbours
France	Andorra,Belgium,Switzerland,Germany,Spain,Italy,Luxembourg,Monaco
Germany	Austria,Belgium,Switzerland,Czechia,Denmark,France,Luxembourg,Netherlands,Poland
Italy	Austria,Switzerland,France,Slovenia,San Marino,Vatican
Russia	Azerbaijan,Belarus,China,Estonia,Finland,Georgia,North Korea,Kazakhstan,Lithuania,Latvia,Mongolia,Norway,Poland,Ukraine
United Kingdom	Ireland

```
1 PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
2 PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
3 PREFIX skos: <http://www.w3.org/2004/02/skos/core#>
4 PREFIX dcterm: <http://purl.org/dc/terms/>
5 PREFIX : <http://example.com/demo/>
6
7 SELECT ?cName1 (GROUP_CONCAT(?cName2;SEPARATOR=",") AS ?neighbours) (COUNT(?cName2) AS ?neighbourCount)
8 WHERE {
9     ?c :country_name ?cName1;
10        :has_country_neighbour/:country_neighbour_value/:country_name ?cName2.
11     { SELECT ?c
12       WHERE {
13         ?c :population ?cPop;
14            :part_of_continent/:continent_name "Europe" .
15       }
16     }
17     ORDER BY DESC(?cPop)
18     LIMIT 5
19 }
20 }
21 GROUP BY ?cName1
22 ORDER BY ASC(?cName1)
```

[Advanced features](#)

Execute Clear

cName1	neighbours	neighbourCount
France	Andorra,Belgium,Switzerland,Germany,Spain,Italy,Luxembourg,Monaco	8
Germany	Austria,Belgium,Switzerland,Czechia,Denmark,France,Luxembourg,Netherlands,Poland	9
Italy	Austria,Switzerland,France,Slovenia,San Marino,Vatican	6
Russia	Azerbaijan,Belarus,China,Estonia,Finland,Georgia,North Korea,Kazakhstan,Lithuania,Latvia,Mongolia,Norway,Poland,Ukraine	14
United Kingdom	Ireland	1

Exercises

List the **names of countries** that start with a capital letter "A" in the continents "Europe" and "North America", ordering the results in ascending alphabetical order.

```
1 PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
2 PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
3 PREFIX skos: <http://www.w3.org/2004/02/skos/core#>
4 PREFIX dcterms: <http://purl.org/dc/terms/>
5 PREFIX : <http://example.com/demo/>
6
7 SELECT ?cName
8 WHERE {
9     ?c :country_name ?cName;
10        :part_of_continent/:continent_name ?ct.
11
12     FILTER(?ct IN ('Europe', 'North America'))
13     FILTER(STRSTARTS(?cName, "A"))
14 }
15
16 ORDER BY ASC(?cName)
```

[Advanced features](#)

cName
Aland Islands
Albania
Andorra
Anguilla
Antigua and Barbuda
Aruba
Austria

Exercises

- For these countries, in a second column in the result table, also list all the names of their associated languages together with any language variant that might be applicable. Each language name and variant pair should be in uppercase and multiple values should be separated by the character "|". For example, if for one of the countries, "Hungarian" was one of the languages with no language variant, and "German" was another associated language with the variant "AT" in the second column, HUNGARIAN|GERMAN(AT) should be displayed.
- Write the query such that the results will be displayed in the table even though the countries might not have associated languages and language variants in the dataset.
- Use the IN function for the two continents in scope, as well as the necessary string functions to tailor the results.

Exercises

```
1 PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
2 PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
3 PREFIX skos: <http://www.w3.org/2004/02/skos/core#>
4 PREFIX dcterms: <http://purl.org/dc/terms/>
5 PREFIX : <http://example.com/demo/>
6
7 SELECT ?cName (GROUP_CONCAT(UCASE(?langWithVariant);SEPARATOR= "|") AS ?alllang)
8 WHERE {
9     ?c :country_name ?cName;
10        :part_of_continent/:continent_name ?ct.
11
12     OPTIONAL{?clang :spoken_in ?c;
13               :language_value/:language_name ?langN.}
14     OPTIONAL {
15         ?clang :language_variant ?langVar .
16     }
17
18     BIND(IF(!BOUND(?langVar), ?langN, CONCAT(?langN, "(", ?langVar, ")")) AS ?langWithVariant)
19     FILTER(?ct IN ('Europe', 'North America'))
20     FILTER(STRSTARTS(?cName, "A"))
21 }
22 GROUP BY ?cName
23 ORDER BY ASC(?cName)
```

[Advanced features](#)

Execute

Clear

cName	alllang
Aland Islands	SWEDISH(AX)
Albania	ALBANIAN GREEK, MODERN (1453-)
Andorra	CATALAN
Anguilla	ENGLISH(AI)
Antigua and Barbuda	ENGLISH(AG)
Aruba	PAPIAMENTO SPANISH DUTCH(AW)
Austria	CROATIAN HUNGARIAN GERMAN(AT)

RDFS classes

```
1 PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
2 PREFIX skos: <http://www.w3.org/2004/02/skos/core#>
3 PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
4 PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
5 PREFIX : <http://example.com/demo/>
6
7 # Class declarations
8 :Currency a rdfs:Class ;
9     skos:prefLabel "currency" @en;
10     rdfs:comment "Manually declared this class"@en ;
11     rdfs:isDefinedBy <http://example.com/demo> .
12
13
14 :Language a rdfs:Class ;
15     skos:prefLabel "Language"@en ;
16     rdfs:comment "Manually declared this class"@en ;
17     rdfs:isDefinedBy <http://example.com/demo> .
```

Automating class creation with SPARQL INSERT

```
1 PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
2 PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
3 PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
4 PREFIX skos: <http://www.w3.org/2004/02/skos/core#>
5 PREFIX : <http://example.com/demo/>
6 # INSERT query to transform types of things into RDFS Classes, in a specific named graph.
7 WITH :General_Country_Info_Graph
8 INSERT {
9   ?t a rdfs:Class ;
10     skos:prefLabel ?tLabelFull ;
11     rdfs:comment "Transformation into RDF Schema performed automatically"@en ;
12     rdfs:isDefinedBy <http://example.com/demo/> .
13 }
14 WHERE {
15   ?s ?p ?o ;
16   a ?t .
17 }
18 BIND(STRLANG(REPLACE(STRAFTER(STR(?t), "http://example.com/demo/"), "[_]", " "), "en") AS ?tLabelFull)
19 }
```

Choose File No file chosen

Type: SPARQL Update Format: Turtle

[Advanced features](#)

Update

totalElapsed=759ms, elapsed=758ms, connFlush=0ms, batchResolve=0, whereClause=473ms, deleteClause=0ms, insertClause=473ms


```
1 PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
2 PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
3 PREFIX skos: <http://www.w3.org/2004/02/skos/core#>
4 PREFIX dcterms: <http://purl.org/dc/terms/>
5 PREFIX : <http://example.com/demo/>
6
7 SELECT DISTINCT ?t
8 WHERE{
9   ?s ?p ?o;
10   a ?t.
11 }
12 ORDER BY ASC(?t)
13
```

[Advanced features](#)

Execute Clear

t
http://example.com/demo/Continent
http://example.com/demo/Country
http://example.com/demo/Country_Language
http://example.com/demo/Country_Neighbour
http://example.com/demo/Currency
http://example.com/demo/Language
rdf:Property
rdfs:Class

RDFS classes

```
1 PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
2 PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
3 PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
4 PREFIX skos: <http://www.w3.org/2004/02/skos/core#>
5 PREFIX dcterms: <http://purl.org/dc/terms/>
6 PREFIX : <http://example.com/demo/>
7
8 SELECT DISTINCT ?c
9 FROM :General_Country_Info_Graph
10 WHERE{
11   ?c a rdfs:Class.
12 }
13 }
14 ORDER BY ASC(?c)
15
```

[Advanced features](#)

Execute

Clear

c
http://example.com/demo/Continent
http://example.com/demo/Country
http://example.com/demo/Country_Language
http://example.com/demo/Country_Neighbour
http://example.com/demo/Currency
http://example.com/demo/Language
rdf:Property

```
1 PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
2 PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
3 PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
4 PREFIX skos: <http://www.w3.org/2004/02/skos/core#>
5 PREFIX dcterms: <http://purl.org/dc/terms/>
6 PREFIX : <http://example.com/demo/>
7
8 DESCRIBE <http://example.com/demo/Continent>
```

[Advanced features](#)

Execute

Clear

subject	predicate	object
http://example.com/demo/Continent	rdf:type	rdfs:Class
http://example.com/demo/Continent	rdfs:comment	Transformation into RDF Schema performed automatically
http://example.com/demo/Continent	rdfs:isDefinedBy	http://example.com/demo
http://example.com/demo/Continent	skos:prefLabel	Continent
http://example.com/demo/Continent/AF	rdf:type	http://example.com/demo/Continent
http://example.com/demo/Continent/AN	rdf:type	http://example.com/demo/Continent
http://example.com/demo/Continent/AS	rdf:type	http://example.com/demo/Continent
http://example.com/demo/Continent/EU	rdf:type	http://example.com/demo/Continent
http://example.com/demo/Continent/NA	rdf:type	http://example.com/demo/Continent
http://example.com/demo/Continent/OC	rdf:type	http://example.com/demo/Continent
http://example.com/demo/Continent/SA	rdf:type	http://example.com/demo/Continent
http://example.com/demo/part_of_continent	rdfs:range	http://example.com/demo/Continent

Automating property creation with SPARQL INSERT

```
1 PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
2 PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
3 PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
4 PREFIX skos: <http://www.w3.org/2004/02/skos/core#>
5 PREFIX : <http://example.com/demo/>
6
7 # INSERT query to transform inter-resource predicates into RDF properties with domain and range specification, in a specific named graph.
8
9 WITH :General_Country_Info_Graph
10 INSERT {
11   ?p a rdf:Property ;
12     rdfs:domain ?st ;
13     rdfs:range ?ot ;
14     skos:prefLabel ?pLabelFull ;
15     rdfs:comment "Transformation into RDF Schema performed automatically"@en ;
16     rdfs:isDefinedBy <http://example.com/demo> .
17 }
18 WHERE {
19   ?s ?p ?o ;
20     a ?st .
21   ?o a ?ot .
22
23   BIND(STRLANG(REPLACE(STRAFTER(STR(?p), "http://example.com/demo/"), "[_]", " "), "en") AS ?pLabelFull)
24
25   FILTER(?p != rdf:type)
26 }
```

Choose File No file chosen

Type: SPARQL Update Format: Turtle

[Advanced features](#)

Update

totalElapsed=390ms, elapsed=380ms, connFlush=0ms, batchResolve=0, whereClause=337ms, deleteClause=0ms, insertClause=337ms

COMMIT: totalElapsed=433ms, commitTime=1711380938017, mutationCount=36

Construct

A more general **CONSTRUCT** query can be created to surface the graph schema (ontology) as a sub-graph. This query can, for example, be used when you don't have a proper schema and want to see what's under-the-hood in terms of the graph vocabulary.

```
1 PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
2 PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
3 PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
4 PREFIX skos: <http://www.w3.org/2004/02/skos/core#>
5 PREFIX : <http://example.com/demo/>
6 CONSTRUCT {
7     ?domain ?p ?range
8 }
9 WHERE {
10     ?s ?p ?o ;
11         a ?domain .
12     OPTIONAL {
13         ?o a ?ot .
14     }
15     BIND(IF(BOUND(?ot), ?ot, DATATYPE(?o)) AS ?range)
16     FILTER(?domain NOT IN(rdfs:Class, rdf:Property))
17     FILTER(?p NOT IN(rdf:type, rdfs:domain, rdfs:range))
18 }
```

subject	predicate	object
http://example.com/demo/Country	http://example.com/demo/area_in_sq_km	xsd:double
http://example.com/demo/Country	http://example.com/demo/population	xsd:integer
http://example.com/demo/Country	http://example.com/demo/part_of_continent	http://example.com/demo/Continent
http://example.com/demo/Country	http://example.com/demo/has_country_neighbour	http://example.com/demo/Country_Neighbour
http://example.com/demo/Country	http://example.com/demo/uses_currency	http://example.com/demo/Currency
http://example.com/demo/Country_Language	http://example.com/demo/rank	xsd:integer
http://example.com/demo/Country_Language	http://example.com/demo/spoken_in	http://example.com/demo/Country
http://example.com/demo/Country_Language	http://example.com/demo/language_value	http://example.com/demo/Language
http://example.com/demo/Country_Language	http://example.com/demo/language_variant	xsd:string
http://example.com/demo/Country	http://example.com/demo/iso_3166-1_numeric_code	xsd:string
http://example.com/demo/Country	http://example.com/demo/capital	xsd:string
http://example.com/demo/Country	http://example.com/demo/country_name	xsd:string
http://example.com/demo/Country	http://example.com/demo/iso_3166-1_alpha-3_code	xsd:string
http://example.com/demo/Country	http://example.com/demo/iso_3166-1_alpha-2_code	xsd:string
http://example.com/demo/Country	skos:prefLabel	rdf:langString
http://example.com/demo/Continent	http://example.com/demo/continent_code	xsd:string
http://example.com/demo/Continent	http://example.com/demo/continent_name	xsd:string
http://example.com/demo/Continent	skos:prefLabel	rdf:langString
http://example.com/demo/Language	http://example.com/demo/language_name	xsd:string
http://example.com/demo/Language	http://example.com/demo/iso_639-3_code	xsd:string
http://example.com/demo/Language	skos:prefLabel	rdf:langString
http://example.com/demo/Currency	http://example.com/demo/currency_name	xsd:string
http://example.com/demo/Currency	http://example.com/demo/currency_code	xsd:string
http://example.com/demo/Currency	skos:prefLabel	rdf:langString
http://example.com/demo/Language	http://example.com/demo/iso_639-1_code	xsd:string
http://example.com/demo/Language	http://example.com/demo/iso_639-2_code	xsd:string
http://example.com/demo/Country_Neighbour	http://example.com/demo/country_neighbour_value	http://example.com/demo/Country

Constructing the graph schema

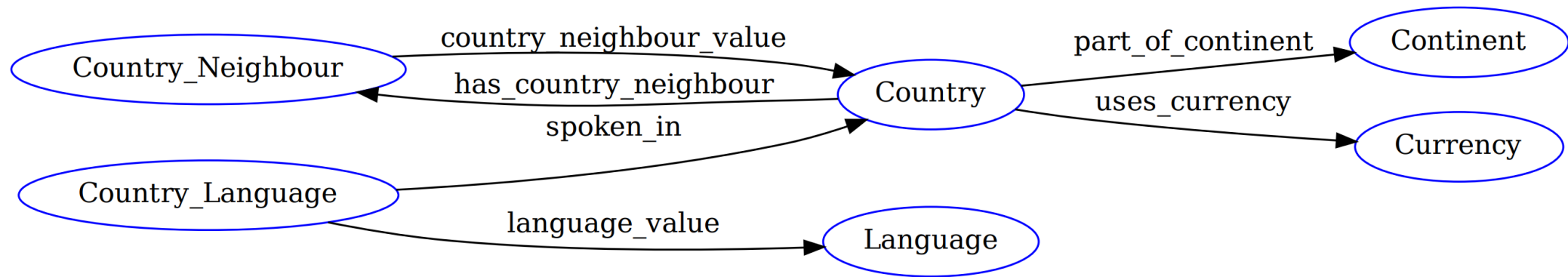
```
1 PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
2 PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
3 PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
4 PREFIX skos: <http://www.w3.org/2004/02/skos/core#>
5 PREFIX : <http://example.com/demo/>
6 CONSTRUCT {
7   ?domain ?property ?range
8 }
9 WHERE {
10  ?property rdfs:domain ?domain ;
11           rdfs:range ?range .
12  FILTER(?domain NOT IN(rdfs:Class, rdf:Property))
13 }
```

[Advanced features](#)

Execute

Clear

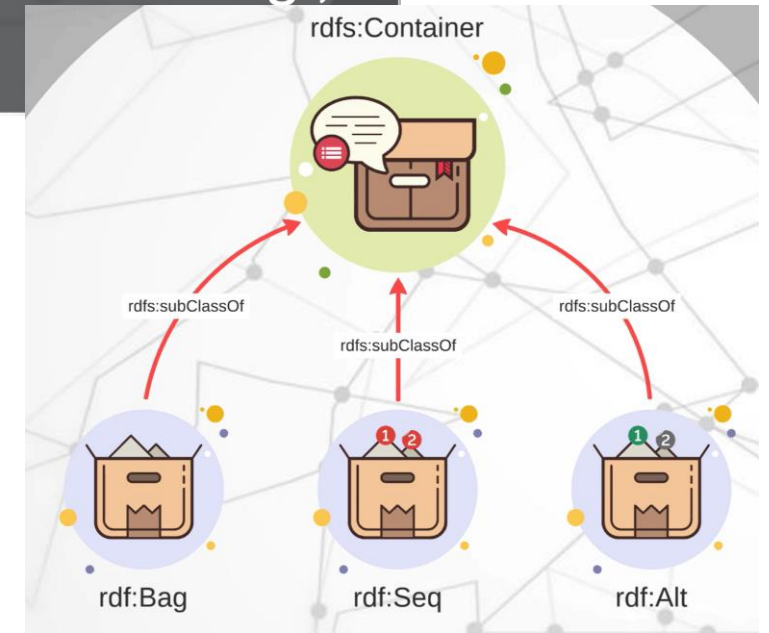
subject	predicate	object
http://example.com/demo/Country_Neighbour	http://example.com/demo/country_neighbour_value	http://example.com/demo/Country
http://example.com/demo/Country	http://example.com/demo/has_country_neighbour	http://example.com/demo/Country_Neighbour
http://example.com/demo/Country_Language	http://example.com/demo/language_value	http://example.com/demo/Language
http://example.com/demo/Country	http://example.com/demo/part_of_continent	http://example.com/demo/Continent
http://example.com/demo/Country_Language	http://example.com/demo/spoken_in	http://example.com/demo/Country
http://example.com/demo/Country	http://example.com/demo/uses_currency	http://example.com/demo/Currency



Namespaces:
rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
xsd: <http://www.w3.org/2001/XMLSchema#>
skos: <http://www.w3.org/2004/02/skos/core#>
<http://example.com/demo/>

RDF Containers and Collections are instance-level resources that allow us to capture lists of things that can be unordered or ordered.

RDF containers are resources used to represent open containers. In RDF, there are three kinds of containers: 'rdf:Bag', 'rdf:Seq' and 'rdf:Alt'.



Declaring and querying RDF Containers

load an instance of `rdf:Bag` and an instance of `rdf:Seq` in two different named graphs.

```
1 PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
2 PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
3 PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
4 PREFIX : <http://example.com/demo/>
5 CREATE GRAPH :Language_Families ;
6 CREATE GRAPH :Travel_Bucket_Lists ;
7 INSERT DATA {
8     GRAPH :Language_Families {
9         :Romance_Languages a rdf:Bag ;
10         rdf:_1 :Language\lat ; # Latin
11         rdf:_2 :Language\ita ; # Italian
12         rdf:_3 :Language\fra ; # French
13         rdf:_4 :Language\spa ; # Spanish
14         rdf:_5 :Language\por ; # Portuguese
15         rdf:_6 :Language\ron ; # Romanian
16         rdf:_7 :Language\cat ; # Catalan
17         rdf:_8 :Language\oci ; # Occitan
18         rdf:_9 :Language\srd ; # Sardinian
19         rdf:_10 :Language\cos . # Corsican
20     }
21     GRAPH :Travel_Bucket_Lists {
22         :My_Travel_Bucket_List a rdf:Seq ;
23         rdf:_1 :Country\NO ; # Norway
24         rdf:_2 :Country\JP ; # Japan
25         rdf:_3 :Country\US ; # US
26         rdf:_4 :Country\TH ; # Thailand
27         rdf:_5 :Country\ZA . # South Africa
28     }
29 }
30
```

Choose File No file chosen

Type: SPARQL Update Format: Turtle

[Advanced features](#)

Update

totalElapsed=5ms, elapsed=0ms, connFlush=0ms, batchResolve=0, whereClause=0ms, deleteClause=0ms, insertClause=0ms

:otalElapsed=5ms, elapsed=0ms, connFlush=0ms, batchResolve=0, whereClause=0ms, deleteClause=0ms, insertClause=0ms

Compose and run a SPARQL query to retrieve from `:Travel_Bucket_Lists` graph and `:General_Country_Info_Graph` as the default graph, the **names of all countries** that start with an uppercase vowel in the container `:My_Travel_Bucket_List`
Also display in the list of results the **index of the countries** in the container as an `xsd:integer`

```
1 PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
2 PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
3 PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
4 PREFIX : <http://example.com/demo/>
5
6 SELECT ?cName ?memberNum
7 FROM :Travel_Bucket_Lists
8 FROM :General_Country_Info_Graph
9 WHERE {
10     VALUES ?v { "A" "E" "I" "O" "U" "Y" }
11
12     :My_Travel_Bucket_List ?p ?o .
13     ?o :country_name ?cName .
14
15     BIND(STR(?p) AS ?pStr)
16     BIND(xsd:integer(STRAFTER(?pStr, "http://www.w3.org/1999/02/22-rdf-syntax-ns#_")) AS ?memberNum)
17
18     FILTER(STRSTARTS(?pStr, STR("http://www.w3.org/1999/02/22-rdf-syntax-ns#_")))
19     FILTER(STRSTARTS(?cName, ?v))
20 }
```

[Advanced features](#)

Execute Clear

cName	memberNum
United States	3

Some tutorial

- <https://atomgraph.github.io/SPARQL-Playground/>
- https://docs.data.world/tutorials/sparql/Your_First_Sparql_Query.html

Python packages

- <https://owlready2.readthedocs.io/en/latest/intro.html>
- RDFLib : <https://rdflib.readthedocs.io/en/stable/index.html>
- Read: <https://www.oclc.org/developer/news/2016/making-sense-of-linked-data-with-python.en.html>

Useful to read

- Database design basics:
- <https://support.microsoft.com/en-gb/office/database-design-basics-eb2159cf-1e30-401a-8084-bd4f9c9ca1f5>
- From SQL Tables To Sparql Graphs ()
- **RDF grapher is a web service for parsing RDF data and visualizing it as a graph:** <https://www.ldf.fi/service/rdf-grapher>
- Stardog: <https://docs.stardog.com/tutorials/rdf-graph-data-model>

- <https://raghavam.github.io/teaching/monsoon2018/ontmodelling/intro-owl.html#/15>